

Задачи с графи

Дизайн и анализ на алгоритми

Теми: DFS, BFS, Minimum Spanning Tree, Shortest Path, Bridges, Articulation Points, SCC

Обхождане в дълбочина (DFS)

Number of Islands

Условие

Дадена е двумерна матрица G с размери $m \times n$, съставена от символите 1 (суша) и 0 (вода). *Остров* се дефинира като максимално свързана област от клетки със стойност 1, свързани хоризонтално или вертикално (не диагонално). Намерете броя на островите в матрицата.

Вход

Матрица G с размери $m \times n$, $G[i][j] \in \{0, 1\}$.

Изход

Едно цяло число — броят на островите.

Сложност

$O(m \cdot n)$

Примери

$$1. \ G = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \Rightarrow 1$$

$$2. \ G = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix} \Rightarrow 3$$

Насока за решение

Предложете алгоритъм, базиран на DFS, който обхожда всяка непосетена клетка със стойност '1' и маркира цялата свързана компонента. обосновете коректността и сложността.

Clone Graph

Условие

Даден е ненасочен свързан граф $G = (V, E)$, представен чрез списък на съседство. Всеки връх $v \in V$ има уникална стойност $\text{val}(v) \in \{1, 2, \dots, |V|\}$. Конструирайте *дълбоко копие* (deep copy) на графа — нов граф със същата структура, но с изцяло нови обекти за всеки връх.

Вход

Референция към произволен връх от графа G . Графът не съдържа примки и кратни ребра.

Изход

Референция към съответния връх в клонирания граф G' .

Сложност

$O(|V| + |E|)$

Примери

1. Граф с 4 върха: $1-2, 2-3, 3-4, 4-1 \Rightarrow$ Нов граф със същата структура, но различни обекти.
2. Граф с 1 връх без ребра $\Rightarrow$ Нов граф с 1 връх без ребра.

Обхождане в ширина (BFS)

Shortest Path in Binary Matrix

Условие

Дадена е двумерна матрица G с размери $n \times n$, съставена от 0 и 1. *Чист път* е път от клетка $(1, 1)$ до клетка (n, n) , при който всички клетки по пътя имат стойност 0. Движението е разрешено в 8 посоки (хоризонтално, вертикално и диагонално). Дължината на пътя е броят на клетките в него. Намерете дължината на най-краткия чист път. Ако такъв път не съществува, върнете -1 .

Вход

Матрица G с размери $n \times n$, $G[i][j] \in \{0, 1\}$.

Изход

Едно цяло число — дължината на най-краткия чист път, или -1 .

Сложност

$O(n^2)$

Примери

1. $G = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \Rightarrow 2$
2. $G = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix} \Rightarrow 4$
3. $G = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix} \Rightarrow -1$ (Стартовата клетка е блокирана.)

Насока за решение

Обяснете защо BFS гарантира намирането на най-краткия път в нетеглен граф. Опишете как моделирате матрицата като граф и как извършвате BFS с 8-посочно съседство.

Rotting Oranges

Условие

Дадена е матрица G с размери $m \times n$, където всяка клетка може да бъде: 0 (празна), 1 (пресен портокал) или 2 (гнил портокал). Всяка минута всеки гнил портокал заразява пресните портокали в четирите си съседни клетки (горе, долу, ляво, дясно). Определете минималния брой минути, след които няма да остане нито един пресен портокал. Ако е невъзможно всички портокали да изгният, върнете -1 .

Вход

Матрица G с размери $m \times n$, $G[i][j] \in \{0, 1, 2\}$.

Изход

Минимален брой минути, или -1 ако остават пресни портокали.

Сложност

$O(m \cdot n)$

Примери

$$1. G = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \Rightarrow 4$$

$$2. G = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \Rightarrow -1 \quad (\text{Долният ляв портокал е недостижим.})$$

$$3. G = \begin{bmatrix} 0 & 2 \end{bmatrix} \Rightarrow 0 \quad (\text{Няма пресни портокали.})$$

Най-кратък път (Shortest Path)

Network Delay Time

Условие

Дадена е мрежа от n възела, номерирани от 1 до n , и m насочени ребра (u_i, v_i, w_i) , означаващи, че сигнал от u_i до v_i се предава за w_i единици време. Изпращате сигнал от възел k . Определете минималното време, за което сигналът ще достигне до *всички* възли. Ако някой възел е недостижим, върнете -1 .

Вход

Списък от m тройки (u_i, v_i, w_i) с $w_i \geq 0$, цяло число n и цяло число k ($1 \leq k \leq n$).

Изход

Минималното време за достигане на сигнала до всички възли, или -1 .

Сложност

$O(m \log n)$ (Dijkstra с приоритетна опашка)

Примери

1. $\text{times} = [(2, 1, 1), (2, 3, 1), (3, 4, 1)], n = 4, k = 2 \Rightarrow 2$
2. $\text{times} = [(1, 2, 1)], n = 2, k = 1 \Rightarrow 1$
3. $\text{times} = [(1, 2, 1)], n = 2, k = 2 \Rightarrow -1$ (Възел 1 е недостижим от възел 2.)

Насока за решение

Приложете алгоритъма на Dijkstra от източник k . Отговорът е $\max_{v \in V} \text{dist}(v)$. Обяснете защо Dijkstra е приложим (неотрицателни тегла) и анализирайте сложността при различни реализации на приоритетната опашка.

Cheapest Flights Within K Stops

Условие

Има n града и m полета, всеки описан с тройка $(\text{from}_i, \text{to}_i, \text{price}_i)$. Намерете най-евтиния маршрут от град src до град dst с **най-много k междинни спирки**. Ако такъв маршрут не съществува, върнете -1 .

Вход

n , списък от m полета, src , dst , k ($0 \leq k < n$), $\text{price}_i \geq 0$.

Изход

Минималната цена на маршрута, или -1 .

Сложност

$O(k \cdot m)$ (модифициран Bellman-Ford)

Примери

1. $n = 4, \text{flights} = [(0, 1, 100), (1, 2, 100), (2, 0, 100), (1, 3, 600), (2, 3, 200)], \text{src} = 0, \text{dst} = 3, k = 1 \Rightarrow 700$ (Маршрут: $0 \rightarrow 1 \rightarrow 3$.)
2. $n = 3, \text{flights} = [(0, 1, 100), (1, 2, 100), (0, 2, 500)], \text{src} = 0, \text{dst} = 2, k = 1 \Rightarrow 200$ (Маршрут: $0 \rightarrow 1 \rightarrow 2$.)
3. $n = 3, \text{flights} = [(0, 1, 100), (1, 2, 100), (0, 2, 500)], \text{src} = 0, \text{dst} = 2, k = 0 \Rightarrow 500$ (Директен полет, без междинни спирки.)

Насока за решение

Опишете модификация на алгоритъма на Bellman-Ford, при която извършвате най-много $k + 1$ итерации на релаксация. Обяснете защо е важно да се използва копие на масива с разстояния при всяка итерация и защо стандартният Dijkstra не е директно приложим.

Find the City With the Smallest Number of Neighbors at a Threshold Distance

Условие

Дадени са n града, номерирани от 0 до $n-1$, и m двупосочни тегловни ребра. Дефинираме: град j е *достижим* от град i в рамките на праг d , ако съществува път от i до j с обща дължина $\leq d$. Намерете града, от който се достигат **най-малко** други градове в рамките на прага d . Ако има няколко такива, върнете този с най-голям номер.

Вход

n , списък от m ребра $[a_i, b_i, w_i]$, праг d , $w_i \geq 1$.

Изход

Номерът на града с най-малко достижими съседи.

Сложност

$O(n^3)$ (Floyd-Warshall) или $O(n \cdot m \log n)$ (Dijkstra от всеки връх)

Примери

1. $n = 4$, edges = $[[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]]$, $d = 4 \Rightarrow 3$ (Град 3 достига само до градове 1 и 2; градове 0, 1, 2 достигат до повече.)
2. $n = 5$, edges = $[[0, 1, 2], [0, 4, 8], [1, 2, 3], [1, 4, 2], [2, 3, 1], [3, 4, 1]]$, $d = 2 \Rightarrow 0$ (Град 0 достига само до град 1.)

Насока за решение

Подход 1: Приложете Floyd-Warshall за намиране на всички двойки най-кратки пътища, след което за всеки град пребройте колко други градове са в рамките на прага. Подход 2: Пуснете Dijkstra от всеки връх. Сравнете двата подхода и обосновайте кой е по-подходящ в зависимост от плътността на графа.

Мостове в граф (Bridges)

Critical Connections in a Network

Условие

Дадена е мрежа от n сървъра, номерирани от 0 до $n-1$, свързани с m ненасочени връзки. Една връзка се нарича *критична*, ако при нейното премахване някои сървъри стават недостижими един от друг, т.е. графът престава да бъде свързан. С други думи, критичните връзки са *мостовете* на графа. Намерете всички критични връзки в мрежата.

Вход

Цяло число n и списък от m двойки $[u_i, v_i]$, описващи ненасочени ребра. Графът е свързан и не съдържа кратни ребра.

Изход

Списък от всички критични връзки (в произволен ред). Всяка връзка се представя като двойка $[u, v]$.

Сложност

$O(n + m)$ (един DFS обход с алгоритъма на Тарян)

Примери

1. $n = 4$, connections = $[[0, 1], [1, 2], [2, 0], [1, 3]] \Rightarrow [[1, 3]]$ (Единствената критична връзка е 1–3; останалите три върха образуват цикъл.)
2. $n = 2$, connections = $[[0, 1]] \Rightarrow [[0, 1]]$ (Единственото ребро е мост.)

Насока за решение

Приложете алгоритъма на Тарян за намиране на мостове. За всеки връх u поддържайте два масива: $\text{disc}[u]$ (времето на откриване при DFS) и $\text{low}[u]$ (минималното disc , достижимо от поддървото на u чрез обратни ребра). Реброто (u, v) е мост тогава и само тогава, когато $\text{low}[v] > \text{disc}[u]$, т.е. от поддървото на v няма обратно ребро, водещо до u или по-нагоре в DFS дървото. Обосновайте защо условието е необходимо и достатъчно и анализирайте сложността.

Minimize Malware Spread II

Условие

Даден е ненасочен граф, представен чрез матрица на съседство G с размери $n \times n$, където $G[i][j] = 1$ означава, че възлите i и j са директно свързани. Даден е също списък `initial` от заразени възли. Разпространението на зловреден софтуер протича така: ако даден възел е заразен, всички възли, свързани с него (пряко или транзитивно), също стават заразени. Можете да **премахнете точно един** възел от списъка `initial` (заедно с всичките му ребра, преди разпространението да започне). Целта е да минимизирате общия брой заразени възли след пълното разпространение. Върнете възела, чието премахване води до най-малко заразени възли. Ако няколко възела дават еднакъв минимален резултат, върнете най-малкия по номер.

Вход

Матрица на съседство G с размери $n \times n$, $G[i][j] \in \{0, 1\}$, и списък `initial` от уникални заразени възли.

Изход

Индексът на възела, който трябва да бъде премахнат.

Сложност

$O(n^2)$

Примери

1. $G = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]$, `initial` = $[0, 1] \Rightarrow 0$
2. $G = [[1, 1, 0], [1, 1, 1], [0, 1, 1]]$, `initial` = $[0, 1] \Rightarrow 1$
3. $G = [[1, 1, 0, 0], [1, 1, 1, 0], [0, 1, 1, 1], [0, 0, 1, 1]]$, `initial` = $[0, 1] \Rightarrow 1$

Насока за решение

Премахването на възел заедно с ребрата му може да раздели свързана компонента на части — това е свързано с концепцията за *срязващи върхове* (articulation points). За всеки кандидат от `initial`: премахнете го от графа, след което с DFS/BFS определете колко възли биха били заразени от останалите възли в `initial`. Изберете кандидата, който води до най-малко заразени. За оптимизация може да се използва Union-Find.

Срязващи върхове (Articulation Points)

Minimum Number of Days to Disconnect Island

Условие

Дадена е двумерна матрица G с размери $m \times n$, съставена от 0 (вода) и 1 (суша). Матрицата съдържа точно един *остров* (максимално свързана област от клетки с 1, свързани хоризонтално или вертикално). Всеки ден можете да промените **точно една** клетка от суша на вода. Целта е да *разделите* острова на две или повече части (или да го премахнете изцяло). Намерете минималния брой дни, необходими за това.

Вход

Матрица G с размери $m \times n$, $G[i][j] \in \{0, 1\}$. Гарантирано е, че има поне една клетка суша.

Изход

Едно цяло число — минималният брой дни (0, 1 или 2).

Сложност

$O(m^2 \cdot n^2)$

Примери

1. $G = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \Rightarrow 2$ (Нужни са 2 дни, за да се раздели квадратът от суша.)
2. $G = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \Rightarrow 0$ (Островът вече е разделен — има повече от една свързана компонента.)
3. $G = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{bmatrix} \Rightarrow 1$ (Средната клетка на втория ред е срязващ връх.)

Насока за решение

Ключовото наблюдение е, че отговорът е винаги 0, 1 или 2. Ако островът вече не е свързан, отговорът е 0. Ако островът съдържа *срязващ връх* (articulation point), тогава премахването на тази клетка разделя острова и отговорът е 1. В противен случай винаги е възможно да се раздели островът с най-много 2 премахвания (премахнете ъглова клетка и нейния единствен съсед). Използвайте алгоритъма на Тарян, за да проверите за наличие на срязващ връх в графа, индуциран от клетките суша.

Critical Connections in a Network (за articulation points)

Условие

Същата задача като описаната по-горе в секция “Мостове” (LeetCode 1192). Алгоритъмът на Тарян за мостове се адаптира лесно за срязващи върхове: връх u е срязващ, ако е коренът на DFS дървото и има ≥ 2 деца, **или** ако u не е корен и съществува дете v , за което $\text{low}[v] \geq \text{disc}[u]$.

Насока за решение

Модифицирайте решението на задачата за мостове: вместо да проверявате $\text{low}[v] > \text{disc}[u]$ (условие за мост), проверявайте $\text{low}[v] \geq \text{disc}[u]$ (условие за срязващ връх). Обърнете специално внимание на коренния връх на DFS дървото.

Силно свързани компоненти (SCC)

Find Eventual Safe States

Условие

Даден е насочен граф с n върха, номерирани от 0 до $n - 1$, представен чрез списъци на съседство: $\text{graph}[i]$ е списъкът от върхове, към които има ребро от i . Един връх се нарича *терминален*, ако няма изходящи ребра. Един връх се нарича *безопасен* (safe), ако **всеки** път, започващ от него, рано или късно достига терминален връх. Намерете всички безопасни върхове, подредени в нарастващ ред.

Вход

Списък на съседство graph с n върха.

Изход

Сортиран списък от всички безопасни върхове.

Сложност

$O(n + m)$, където m е броят на ребрата

Примери

- $\text{graph} = [[1, 2], [2, 3], [5], [0], [5], [], []] \Rightarrow [2, 4, 5, 6]$ (Върхове 5 и 6 са терминални; 2 и 4 водят до тях. Върхове 0, 1, 3 участват в цикъл.)
- $\text{graph} = [[1, 2, 3, 4], [1, 2], [3, 4], [0, 4], []] \Rightarrow [4]$ (Само връх 4 е терминален и безопасен.)

Насока за решение

Върховете, които са *небезопасни*, са точно тези, които участват в цикъл или могат да достигнат цикъл. Използвайте DFS с три състояния на посещение (бял/сив/черен) за откриване на цикли. Алтернативен подход: намерете всички SCC; върховете в SCC с повече от един елемент (или със самопримка) са небезопасни, както и тези, от които е достижима такава SCC.

Course Schedule

Условие

Има n курса, номерирани от 0 до $n - 1$. Даден е списък от m зависимости (prerequisites): двойка $[a_i, b_i]$ означава, че за да вземете курс a_i , трябва първо да сте завършили курс b_i . Определете дали е възможно да завършите всички n курса, т.е. дали не съществува цикъл в графа на зависимостите.

Вход

Цяло число n и списък от m двойки $[a_i, b_i]$.

Изход

true, ако е възможно да се завършат всички курсове; **false** в противен случай.

Сложност

$O(n + m)$

Примери

1. $n = 2$, prerequisites = $[[1, 0]] \Rightarrow \text{true}$ (Вземете курс 0, после курс 1.)
2. $n = 2$, prerequisites = $[[1, 0], [0, 1]] \Rightarrow \text{false}$ (Цикъл: курсове 0 и 1 зависят взаимно.)

Насока за решение

Задачата се свежда до проверка за наличие на цикъл в насочен граф. Подход 1: DFS с три цвята (бял/сив/черен) — ако срещнем сив връх, има цикъл. Подход 2: Алгоритъм на Кан (BFS-базирано топологично сортиране) — ако не всички върхове влязат в топологичната наредба, има цикъл. И двата подхода са свързани с концепцията за SCC: граф без цикъл е DAG, в който всяка SCC съдържа точно един връх.

Course Schedule II

Условие

Същата настройка като **Course Schedule**: n курса и списък от зависимости. Този път трябва да върнете *валидна наредба* за завършване на всички курсове (топологично сортиране). Ако съществува цикъл и е невъзможно да се завършат всички курсове, върнете празен списък.

Вход

Цяло число n и списък от m двойки $[a_i, b_i]$.

Изход

Списък с наредба на курсовете, или празен списък ако е невъзможно.

Сложност

$O(n + m)$

Примери

1. $n = 2$, prerequisites = $[[1, 0]] \Rightarrow [0, 1]$
2. $n = 4$, prerequisites = $[[1, 0], [2, 0], [3, 1], [3, 2]] \Rightarrow [0, 1, 2, 3]$ или $[0, 2, 1, 3]$ (Множество валидни наредби.)
3. $n = 1$, prerequisites = $[] \Rightarrow [0]$

Насока за решение

Приложете DFS-базирано или BFS-базирано (Кан) топологично сортиране. При DFS подхода: след като обработите всички съседни на даден връх, добавете го в стек; крайният ред е обратният на стека. При BFS подхода: започнете от върховете с входяща степен 0 и последователно ги премахвайте. Обяснете връзката с SCC: топологично сортиране на кондензацията на графа (графа от SCC) винаги е възможно.

Number of Operations to Make Network Connected

Условие

Дадени са n компютъра, номерирани от 0 до $n - 1$, и списък от m Ethernet кабели, всеки свързващ два компютъра. Можете да **преместите** кабел от една връзка към друга (т.е. да премахнете съществуващо ребро и да го добавите другаде). Определете минималния брой премествания, необходими за свързване на всички компютри в една мрежа. Ако няма достатъчно кабели (т.е. $m < n - 1$), върнете -1 .

Вход

Цяло число n и списък от m двойки $[a_i, b_i]$.

Изход

Минималният брой премествания, или -1 .

Сложност

$O(n + m)$

Примери

1. $n = 4$, connections = $[[0, 1], [0, 2], [1, 2]] \Rightarrow 1$ (Преместете кабела 1–2 (излишен, защото образува цикъл към 1–3 или 2–3.)
2. $n = 6$, connections = $[[0, 1], [0, 2], [0, 3], [1, 2], [1, 3]] \Rightarrow 2$ (Има 2 излишни кабела и 3 свързани компоненти.)
3. $n = 6$, connections = $[[0, 1], [0, 2], [0, 3], [1, 2]] \Rightarrow -1$ (4 кабела $< 6 - 1 = 5$ — не стигат.)

Насока за решение

Необходимо условие е $m \geq n - 1$. Ако е изпълнено, отговорът е (брой свързани компоненти -1), тъй като всеки “излишен” кабел (ребро, което затваря цикъл) може да се премести, за да свърже две компоненти. Използвайте DFS/BFS или Union-Find за броене на свързаните компоненти.

Отрицателни цикли (Negative Cycle Detection)

Currency Exchange Arbitrage

Условие

Дадена е валутна борса с n валути, номерирани от 0 до $n - 1$, и списък от m обменни курса. Всеки обменен курс е тройка (u_i, v_i, r_i) , означаваща, че за 1 единица от валута u_i получавате $r_i > 0$ единици от валута v_i . Започвате с 1,00 долар (валута 0). Определете дали съществува последователност от обмени, чрез която можете да се върнете обратно до долари и да получите **строго повече** от 1,00 долар. С други думи, проверете дали в графа на обменните курсове съществува цикъл, преминаващ през валута 0, при който произведението на курсовете по цикъла е строго по-голямо от 1.

Вход

Цяло число n , списък от m тройки (u_i, v_i, r_i) с $r_i > 0$.

Изход

true, ако съществува печеливш цикъл, достижим от валута 0; **false** в противен случай.

Сложност

$O(n \cdot m)$ (Bellman-Ford)

Примери

1. $n = 3$, rates = $[(0, 1, 0,9), (1, 2, 0,8), (2, 0, 1,4)] \Rightarrow \text{true}$ (Цикъл: $1 \xrightarrow{\times 0,9} 0,9 \xrightarrow{\times 0,8} 0,72 \xrightarrow{\times 1,4} 1,008 > 1$.)
2. $n = 3$, rates = $[(0, 1, 0,9), (1, 2, 0,8), (2, 0, 1,3)] \Rightarrow \text{false}$ (Цикъл: $0,9 \times 0,8 \times 1,3 = 0,936 < 1$.)
3. $n = 4$, rates = $[(0, 1, 1,2), (1, 2, 1,3), (2, 3, 0,5), (3, 1, 1,8), (1, 0, 0,7)] \Rightarrow \text{true}$ (Цикъл $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$: $1,3 \times 0,5 \times 1,8 = 1,17 > 1$, и е достижим от валута 0.)

Насока за решение

Задачата се свежда до търсене на *отрицателен цикъл* в подходящо трансформиран граф. Произведението на курсовете по цикъл е $r_1 \cdot r_2 \cdots r_k > 1$. Логаритмуваме: $\log r_1 + \log r_2 + \cdots + \log r_k > 0$, или еквивалентно $(-\log r_1) + (-\log r_2) + \cdots + (-\log r_k) < 0$. Следователно заменяме теглото на всяко ребро (u, v, r) с $w(u, v) = -\log r$ и търсим *отрицателен цикъл*, достижим от връх 0, с алгоритъма на Bellman-Ford. Ако след $n - 1$ итерации на релаксация все още може да се подобри някое разстояние до връх, достижим от 0, тогава съществува отрицателен цикъл и отговорът е **true**. Обосновете коректността на логаритмичната трансформация и анализирайте сложността.

За всяка задача се очаква: описание на алгоритъма стъпка по стъпка, доказателство за коректност и анализ на времева сложност

Контакти

При въпроси, свързани с тези или други задачи, може да се свържете с мен тук:

Instagram: [instagram.com/stanislav._.igov](https://www.instagram.com/stanislav._.igov)

Facebook: [facebook.com/stanislav.igov](https://www.facebook.com/stanislav.igov)